WEEK 3 · LESSON 15 OF 24 · HANDOUT

Explaining Model Decisions

Study time: **about 55 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=Wl9L592ycc4 — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyCon US 2019 tutorial: Open the Black Box — an Introduction to Model Interpretability in Python — https://www.youtube.com/watch?v=d66ttkMY9zE

🎓 **Go deeper:** PyData Amsterdam 2019: SHAP and Beyond — https://www.youtube.com/watch?v=xwl8WhtJNs0

Your goal in this lesson: Explain what drives a model overall and why it made one decision, give people clear reasons and actionable "what would change the decision" answers, and know the limits of explanations.

Who needs explanations?

- **The applicant**: "Why was I refused, and what can I do?"
- **The loan officer**: "Can I trust this score?"
- **The developer and auditor**: "Is the model using sensible signals?"
- **The regulator**: "Can you justify decisions that affect people?"

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]
TARGET = "repaid"
train, test = train_test_split(applicants, test_size=0.25, random_state=0,
stratify=applicants[TARGET])
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(train[FEATURES], train[TARGET])
proba = model.predict_proba(test[FEATURES])[:, 1]
print(f"accuracy {accuracy_score(test[TARGET], proba >= 0.5):.3f} | AUC
{roc_auc_score(test[TARGET], proba):.3f}")
```

Global explanations: what matters overall

```
from sklearn.inspection import permutation_importance

coefs = pd.Series(model[-1].coef_[0], index=FEATURES)                  # on standardised
features
perm = permutation_importance(model, test[FEATURES], test[TARGET], scoring="roc_auc",
n_repeats=10, random_state=0)
importance = pd.DataFrame({"weight (standardised)": coefs.round(2),
                           "AUC lost when shuffled": pd.Series(perm.importances_mean,
index=FEATURES).round(3)})
print(importance.sort_values("AUC lost when shuffled", ascending=False).to_string())
```

- **Weights** of a logistic regression on standardised data show direction (+ helps repayment, − hurts) and strength.
- **Permutation importance** works for **any** model: shuffle one column and see how much the score drops. A big drop means the model relies on it.

Check these against common sense: income, loan size and late payments should matter most — and they do.

Local explanations: why this applicant?

For a logistic regression, each feature's contribution to the score is simply weight × (standardised value). Compared with an average applicant, that gives clear **reason codes**:

```
scaler, lr = model[0], model[-1]
REASONS = {"income_rwf": "income", "loan_rwf": "loan amount", "years_member": "years as a
member",
           "late_payments": "late payments", "salaried": "salary status", "momo_tx_month":
"mobile-money activity"}

def explain(applicant, top=2):
    row = pd.DataFrame([applicant])[FEATURES]
    p = float(model.predict_proba(row)[0, 1])
    contrib = pd.Series(lr.coef_[0] * scaler.transform(row)[0], index=FEATURES)
    hurts = contrib.sort_values().head(top)
    helps = contrib.sort_values(ascending=False).head(top)
    print(f"repay probability {p:.0%} -> {'APPROVE' if p >= 0.5 else 'REFER / DECLINE'}")
    print("  main reasons against:", ", ".join(f"{REASONS[f]} ({v:+.2f})" for f, v in
hurts.items() if v < 0) or "none")
    print("  main reasons for:    ", ", ".join(f"{REASONS[f]} ({v:+.2f})" for f, v in
helps.items() if v > 0) or "none")

aline = {"income_rwf": 180_000, "loan_rwf": 520_000, "years_member": 2, "late_payments": 2,
"salaried": 0, "momo_tx_month": 20}
explain(aline)
```

Counterfactuals: what would change the decision?

People find "what could I change?" more useful than numbers:

```
def smallest_change(applicant, feature, values):
    for v in values:
        trial = dict(applicant, **{feature: v})
        if model.predict_proba(pd.DataFrame([trial])[FEATURES])[0, 1] >= 0.5:
            return v
    return None

loan_ok = smallest_change(aline, "loan_rwf", range(520_000, 0, -10_000))
late_ok = smallest_change(dict(aline, loan_rwf=300_000), "late_payments", [1, 0])
print(f"Approved if the loan were {loan_ok:,} RWF or less (instead of 520,000)." if loan_ok
else "No loan amount works alone.")
print(f"With a 300,000 RWF loan: approved if late payments drop to {late_ok}." if late_ok is
not None
      else "With a 300,000 RWF loan, fewer late payments alone would not be enough.")
```

A good message to Aline: "We could approve a loan of up to the amount above today. Paying on time for the next months would also improve your result." Only suggest changes a person **can** make — never "change your age or area".

SHAP and the limits of explanations

For complex models (forests, boosting, neural networks), the **SHAP** library computes similar per-feature contributions using game theory. Limits to remember:

- An explanation describes the **model**, not the real world: "late payments" explains the score, it does not prove causation.
- Explanations can be **misleading** for correlated features (income and salary move together).
- A clear explanation of an unfair model is still an unfair model.
- Simple, explainable models are often nearly as accurate (Lesson 8) — prefer them for high-stakes decisions.

Key terms

- **Global explanation** — what drives the model overall.
- **Local explanation** — why the model gave one particular answer.
- **Permutation importance** — score lost when one feature is shuffled.
- **Reason codes** — the top factors behind a decision, in plain words.
- **Counterfactual explanation** — the smallest realistic change that would change the decision.
- **SHAP** — a method and library for per-feature contributions in any model.

Remember this: Explain models globally and individually, give people reasons and realistic counterfactuals in plain language, and remember that explanations describe the model, not the truth.

Name: _____ Date: _____

Activity

Task 1

Explain three applicants of your own design, including one who is approved.

Task 2

Find the smallest increase in `years_member` that would approve Aline (keep the 520,000 loan).

Task 3

Rewrite the reason codes as one friendly SMS of under 160 characters.

Task 4

Compute permutation importance with accuracy instead of AUC. Does the order change?

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does permutation importance measure?
   - A. Model size
   - B. The number of features
   - C. How much the score drops when one feature is shuffled
   - D. Training time
2. What is a local explanation?
   - A. A server log
   - B. Why the model gave one particular answer
   - C. A summary of all data
   - D. An explanation in the local language only
3. What is a counterfactual explanation?
   - A. The smallest realistic change that would change the decision
   - B. A random example
   - C. A false statement
   - D. A legal document
4. Which change should never be suggested to an applicant?
   - A. Pay on time
   - B. Borrow less
   - C. Save more
   - D. Change your age or area
5. What is a limit of explanations?
   - A. They are always causal
   - B. They describe the model, not proof of cause in the real world
   - C. They make models fair
   - D. They need no data
6. What does SHAP do?
   - A. Computes per-feature contributions for any model
   - B. Trains models faster
   - C. Draws maps
   - D. Encrypts data

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.